NCCLPol: Verified, Composable Policy Execution for GPU Collective Communication

Anonymous Author(s)

ABSTRACT

NCCL is the de facto standard library for GPU collective communication in large-scale distributed training. It exposes a plugin system—tuner, profiler, net, and env plugins—that allows operators to customize collective behavior. However, these plugins execute as native code within NCCL's address space, with no isolation, no verification, and no safe mechanism for cross-plugin state sharing. A single null-pointer dereference in a tuner plugin crashes the entire training job, potentially wasting thousands of GPU-hours.

We present NCCLPol, a system that embeds a userspace eBPF runtime inside NCCL's plugin interface to transform unsafe extension points into verified, composable policy hooks. NCCLPol introduces three key mechanisms: (1) load-time static verification that rejects unsafe policies before they can affect any collective decision, (2) typed eBPF maps that enable structured state sharing between profiler and tuner without ad-hoc shared memory, and (3) atomic policy hot-reload that swaps policies without losing or corrupting in-flight collective calls. All of this operates entirely within NCCL's stock plugin ABI, requiring zero modifications to NCCL itself. Our evaluation on real NCCL workloads shows that NCCLPol adds only 42–70 ns of CPU overhead per collective decision (well within NCCL's latency budget), rejects all seven tested classes of unsafe programs at load time, and enables closed-loop profiler-to-tuner adaptation that is difficult and error-prone with native plugins alone.

1 INTRODUCTION

Large-scale GPU training workloads depend critically on collective communication performance. A single poorly-configured `AllReduce` can bottleneck an entire training pipeline, and the choice of algorithm (ring vs. tree), protocol (LL vs. Simple), and channel count can shift collective latency by orders of magnitude [2, 3, 6]. NVIDIA's NCCL library [6] has become the dominant runtime for this task, supporting multi-GPU and multi-node topologies across InfiniBand, RoCE, and shared-memory transports.

Recognizing that no single configuration fits all deployments, NCCL exposes a *plugin system*: tuner plugins override algorithm and protocol selection per collective call; profiler plugins receive timestamped event callbacks; net plugins control network device selection and connection management; and env plugins programmatically set runtime parameters. This extensibility has enabled custom tuning solutions at major cloud providers and in research prototypes [2, 3].

The safety gap. However, NCCL's extension model has a fundamental flaw: *every plugin executes as native code in NCCL's address space with zero safety guarantees.* There is no verification that a tuner plugin terminates, no bounds checking on context access, no isolation between plugin and library state, and no structured mechanism for plugins to share state (e.g., for a profiler to inform a tuner). A single bug—a null-pointer dereference, an out-of-bounds read, a division by zero—crashes the hosting process with a `SIGSEGV`, terminating the training job. In production GPU clusters where jobs run for days across thousands of GPUs, such crashes waste enormous resources and are notoriously difficult to diagnose because the crash occurs inside a `dlopen`'d shared object with no structured error reporting.

The situation worsens when operations teams manage plugin deployments across a fleet. Operators need to roll out, roll back, and iterate on tuning policies across heterogeneous clusters—but loading native code plugins provides no safety net against bugs introduced during these updates. The result is a stark choice: either grant full native-code access (unsafe) or deny plugin customization entirely (inflexible). While we do not have deployment-scale failure data for NCCL plugins specifically, these failure modes are well-known from decades of experience with kernel modules, device drivers, and other native-code extension mechanisms. As model architectures diverge (e.g., mixture-of-experts vs. dense transformers have fundamentally different communication patterns), the need for safe, updatable tuning policies grows.

Our insight. We observe that the eBPF execution model—load-time static verification, bounded execution, typed map-based state sharing—is a natural fit for NCCL's plugin architecture. The key enabling observation is that NCCL's policy-critical hot path, `getCollInfo()`, executes on the *CPU side* and is *latency-tolerant*: the collective communication it configures takes microseconds to milliseconds on the GPU, making an additional 50–80 ns of CPU-side decision overhead negligible. This contrasts sharply with eBPF's traditional domain (packet processing, syscall filtering) where nanosecond budgets are tight. In collective communication, eBPF's overhead budget is generous, and the safety and composability benefits are large.

Moreover, eBPF's map abstraction directly addresses NCCL's cross-plugin communication gap. In stock NCCL, the profiler and tuner plugins are loaded independently via `dlopen` and have no shared state mechanism. An operator who wants profiler-informed tuning must implement ad-hoc shared memory, inter-plugin signaling, and custom synchronization—all of which are error-prone and unverifiable. eBPF maps provide exactly the right abstraction: typed, concurrent, safe state sharing between programs, with atomic access semantics enforced by the runtime.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

System overview. NCCLPol embeds a userspace eBPF runtime (bpftime [11]) inside a standard NCCL tuner plugin. Policy authors write eBPF programs in restricted C that are compiled to BPF bytecode, statically verified at load time, and JIT-compiled to native code for execution. Cross-plugin state sharing occurs exclusively through typed eBPF maps—for example, the profiler plugin writes latency observations into a map that the tuner policy reads when making algorithm decisions. Policy hot-reload atomically swaps the active policy via a single pointer exchange, ensuring zero lost calls. The entire system operates within NCCL's stock plugin ABI: NCCLPol is loaded via standard `NCCL_TUNER_PLUGIN` and `NCCL_PROFILER_PLUGIN` environment variables, requiring no modifications to NCCL itself.

Contributions. This paper makes the following contributions:

(1) **The case for verified extensibility in collective communication.** We identify a fundamental tension in NCCL's plugin model between customizability and safety, and show that the eBPF execution model resolves this tension with negligible overhead because collective communication's CPU-side decision path is inherently latency-tolerant (§2).
(2) **A map-based composability model for cross-plugin state.** We demonstrate that eBPF's typed maps provide a principled mechanism for closed-loop control—profiler-informed tuning—that is difficult and error-prone with native plugins' ad-hoc shared memory (§3).
(3) **Atomic policy evolution without call loss.** We design and implement a hot-reload mechanism that atomically swaps policies with respect to in-flight collective decisions, achieving a 0.309 µs swap window with zero lost calls across 400,000 active invocations (§3).
(4) **Evaluation on real NCCL deployments.** We evaluate NCCLPol on NCCL 2.29.7 with real GPU hardware (RTX 5090), demonstrating 42–70 ns CPU overhead, verifier coverage of common unsafe patterns (7/7 tested classes rejected), and end-to-end GPU overhead within measurement noise (§5).

2 BACKGROUND AND MOTIVATION

2.1 NCCL Plugin Architecture

NCCL [6] organizes its extensibility around four plugin types, each loaded via `dlopen` at initialization time. Understanding their interfaces is essential to appreciating both the opportunity and the safety gap that NCCLPol addresses.

Tuner plugin. The tuner plugin's `getCollInfo()` function is called once per collective operation. It receives the collective type (AllReduce, AllGather, etc.), message size, number of ranks, and current topology information, and returns the algorithm (ring, tree, collnet), protocol (LL, LL128, Simple), and number of channels to use. This is the primary policy decision point in NCCL. The tuner API (v3 as of NCCL 2.29) provides cost arrays for algorithms and protocols; the tuner sets costs to zero for preferred choices and to a large sentinel value for non-preferred ones, allowing NCCL to fall back gracefully.

Profiler plugin. The profiler plugin receives structured event callbacks—group start/end, collective start/end, proxy start/end—with nanosecond timestamps. It is NCCL's telemetry interface, designed for performance monitoring and debugging. However, the profiler provides no built-in mechanism to feed observations back to the tuner or any other plugin.

Net plugin. The net plugin controls network device enumeration, connection setup, and data transfer operations. It is used for custom transport implementations (e.g., NVIDIA SHARP, AWS EFA, Azure Infiniband) and provides fine-grained control over RDMA resources, memory registration, and communication buffers.

Env plugin. Introduced in NCCL 2.28, the env plugin allows programmatic parameter setting without environment variables, enabling per-communicator configuration.

The gap. These plugins are designed as independent extension points. NCCL provides no mechanism for cross-plugin communication: a profiler cannot pass latency measurements to a tuner, a tuner cannot query network state from the net plugin, and no plugin can safely share state with another. Each plugin is a shared object loaded into NCCL's address space with full native-code privileges. Furthermore, there is no mechanism to update a plugin without restarting the NCCL communicator (and thus the training job), no verification of plugin correctness, and no isolation between plugin and NCCL internal state.

2.2 The Cost of Unsafe Extensions

To concretize the safety problem, consider three failure scenarios that arise with native NCCL plugins:

Scenario 1: Null dereference. A tuner plugin accesses a data structure that has been freed or was never initialized. The result is an immediate `SIGSEGV`. The hosting training process terminates. On a 1,024-GPU training job running for 72 hours, this wastes potentially thousands of GPU-hours. The crash backtrace points into a `dlopen`'d `.so` file, making root-cause analysis difficult—the plugin has no structured error reporting, and the crash may occur deep in a third-party library linked into the plugin.

Scenario 2: Silent corruption. A tuner plugin has an off-by-one error that overwrites an adjacent NCCL internal structure. The corruption does not cause an immediate crash but produces subtly incorrect algorithm selections. Training proceeds but converges slowly or produces incorrect gradients. This failure mode is worse than a crash because it is *silent*: the operator sees degraded performance or incorrect results but has no indication that the tuner plugin is the cause. Debugging requires binary bisection across plugin versions—a process that can take days on production clusters.

Scenario 3: Infinite loop. A tuner plugin enters an infinite loop due to a logic error (e.g., a convergence check that never terminates). Because `getCollInfo()` is called synchronously from NCCL's collective launch path, the entire NCCL communicator hangs. All ranks in the collective are blocked indefinitely. No timeout mechanism exists within NCCL's plugin interface, and the hang is indistinguishable from a network deadlock without detailed profiling.

In all three scenarios, the root cause is the same: native plugins execute with the full privileges of the hosting process and no safety net. The `dlopen`-based extension model provides maximum flexibility at the cost of zero safety.

2.3 Why eBPF Fits

The eBPF execution model [9] was designed precisely to allow safe, verified extension of privileged software. An eBPF program undergoes static verification before execution: the verifier checks for null dereferences, out-of-bounds accesses, unbounded loops, uninitialized reads, and illegal helper function calls. Programs that fail verification are rejected before they can execute a single instruction.

Three properties make eBPF particularly well-suited to NCCL's plugin architecture:

1. Generous overhead budget. In NCCL, `getCollInfo()` is called once per collective operation on the CPU. The collective operation itself—the actual GPU-side data movement—takes microseconds to milliseconds (e.g., 175 µs for a 128 MB AllReduce on our hardware). An additional 50–80 ns of CPU overhead for eBPF dispatch and JIT execution is negligible—less than 0.05% of collective latency. Even for the smallest collectives (1 KB messages, ∼5 µs latency), the overhead is under 1.5%. This is a fundamentally different overhead regime than eBPF's traditional domains: in XDP packet processing, every nanosecond of per-packet overhead matters; in NCCL policy decisions, 80 ns is invisible.

2. Natural state-sharing primitive. eBPF maps (hash maps, arrays, ring buffers) provide typed, concurrent, structured state sharing between programs. This directly addresses NCCL's cross-plugin communication gap: a profiler eBPF program writes latency observations into a map; a tuner eBPF program reads the same map when making decisions. No ad-hoc shared memory, no locking bugs, no type confusion. The map abstraction also provides natural per-communicator state separation: a hash map keyed by communicator ID ensures that one communicator's policy state is logically separated from another's, though all maps still reside in the same process address space.

3. Hot-reload without restart. eBPF programs can be atomically replaced at runtime. For NCCL, this means operators can update tuning policies without restarting training jobs. While native `dlopen`-based plugins could in principle support hot reload through custom indirection layers, eBPF provides this capability as a built-in primitive with verification of the replacement program before swap. In long-running training jobs, this enables policy iteration without losing training progress.

2.4 Comparison with Prior Approaches

Existing work on collective communication optimization falls into two categories, neither of which addresses the safety gap:

Collective algorithm synthesis [2, 3, 8] focuses on generating efficient communication schedules for specific topologies. These systems produce static plans that are compiled ahead of time; they do not address runtime policy selection, safety, or composability. A synthesized schedule from MSCCL [2] still needs a tuner plugin to activate it at runtime—and that tuner plugin runs as unsafe native code.

eBPF in systems software [5, 10, 12, 13] has been applied to storage (XRP [12]), distributed protocols (Electrode [13]), networking (eTran [5]), and memory management (PageFlex [10]). However, no prior work applies eBPF to GPU collective communication runtimes. NCCLPol is the first system to bring verified, composable policy execution to this domain.

3 DESIGN

NCCLPol's design is guided by four principles that address the problems identified in §2. We first state the principles, then describe the architecture and programming model that realize them.

3.1 Design Tensions

NCCLPol's design navigates four tensions inherent in adding verified extensibility to a performance-critical runtime. Each tension shaped a concrete design choice.

T1: Safety vs. overhead. Verifying policies at every invocation would guarantee that only safe code runs, but would add milliseconds of overhead per collective call—clearly unacceptable. Verifying only at load time trades per-call dynamic checks for a one-time verification cost (1–5 ms), amortized over the training job's lifetime (hours to days). This design relies on the invariant that verified BPF bytecode, once JIT-compiled, cannot violate its safety guarantees at runtime—a property ensured by the eBPF execution model's lack of dynamic code generation, unbounded allocation, and arbitrary memory access. The tradeoff is that verification cannot catch runtime-dependent bugs (e.g., a policy that is safe but produces suboptimal decisions); it guarantees memory safety and termination, not semantic correctness.

T2: Structured sharing vs. flexibility. All cross-component state sharing occurs through typed eBPF maps. Maps enforce type safety and provide atomic access semantics, making the data-sharing contract between profiler and tuner explicit and inspectable. The tradeoff is expressiveness: maps constrain policies to fixed-size keys and values with a predefined set of operations (lookup, update, delete). Policies cannot build arbitrary data structures, pass pointers between plugins, or use custom synchronization. We accept this constraint because NCCL policy state is inherently simple (per-communicator scalars: latency, channel count, algorithm choice), and the structured sharing eliminates the locking bugs and type confusion that plague ad-hoc shared memory in native plugins.

T3: Availability vs. consistency during updates. Policy hot-reload prioritizes availability: when an operator deploys a new policy, the system verifies it, JIT-compiles it, and atomically swaps the function pointer. Any `getCollInfo()` call in progress completes under the old policy; the next call executes under the new one. No call is lost, and no call sees a partially-initialized policy. If the new policy fails verification, the old policy continues uninterrupted. The tradeoff is that there is no synchronized transition across all threads or ranks—different threads may briefly execute different policies. We accept this because NCCL collective decisions are independent per call, and a brief policy mix during transitions does not affect correctness.

T4: Compatibility vs. capability. NCCLPol operates entirely within NCCL's existing plugin interface, loaded as a standard tuner and profiler plugin via environment variables. No NCCL source modifications, no custom builds, no kernel modules. The tradeoff is that the action space is limited to what the stock ABI exposes: algorithm, protocol, and channel count selection via cost arrays. Deeper interventions—modifying NCCL's internal transport selection, injecting custom communication kernels, or controlling memory registration—are outside the current scope. We accept this limitation because drop-in deployability enables adoption without coordinating NCCL version upgrades, and the exposed action space already covers the most impactful tuning knobs.

3.2 Threat Model and Safety Scope

NCCLPol targets a specific threat model that should be stated explicitly.

Policy authorship. Policies are written by *platform operators* or *semi-trusted users*—infrastructure engineers who manage GPU clusters and need to deploy, update, and roll back tuning configurations safely. NCCLPol does **not** assume fully untrusted tenants submit arbitrary BPF code; in a multi-tenant setting, the platform operator writes policies that differentiate tenant behavior based on communicator identity.

What is verified. The eBPF verifier (PREVAIL-based static analysis) checks: (1) memory safety—no null dereferences, no out-of-bounds context or map accesses; (2) bounded execution—all loops have provable bounds, preventing hangs; (3) stack safety—no writes beyond the 512-byte BPF stack; and (4) helper whitelist—only registered map operations can be called, no arbitrary function pointers or system calls.

*What is **not** verified.* The system does not protect against: (1) *resource exhaustion*—a policy that allocates many map entries or performs expensive (but bounded) computation consumes CPU and memory within the host process; (2) *side channels*—timing or cache-based information leakage between policies sharing the same address space; (3) *bugs in the host runtime*—the bpftime JIT, PREVAIL verifier, map implementation, and native host plugin are part of the trusted computing base (TCB); (4) *semantic correctness*—the verifier ensures a policy terminates and does not crash, but cannot ensure it makes good tuning decisions.

Security posture. NCCLPol provides *defense in depth*, not complete sandboxing. The verifier eliminates the most common and costly failure modes (crashes, hangs, memory corruption) that arise from plugin bugs. It does not replace process-level isolation for truly untrusted code. This is analogous to the Linux kernel's eBPF model, where the verifier prevents kernel crashes from BPF programs but does not defend against all possible denial-of-service attacks.

3.3 Architecture Overview

Figure 1 illustrates NCCLPol's architecture. The system has four components, each addressing a specific design requirement:

Policy compiler toolchain. Policy authors write eBPF programs in restricted C. The programs are compiled with `clang -target bpf` to produce standard BPF ELF objects. The restricted C subset enforces bounded loops, no dynamic allocation, no function pointers, and access to only declared helper functions and map types. This is the same programming model used for Linux kernel eBPF programs, ensuring familiarity for systems programmers. The toolchain requires only standard LLVM/Clang, with no custom compiler or language extensions.

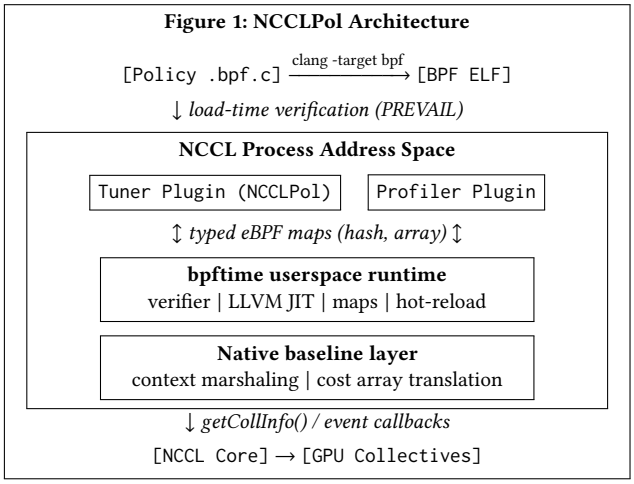

Figure 1: NCCLPol architecture. Policy programs are compiled to BPF bytecode, verified at load time via PREVAIL, and JIT-compiled by bpftime's LLVM backend inside the NCCL process. The tuner and profiler plugins communicate through shared typed eBPF maps, enabling closed-loop adaptation. A native baseline layer handles NCCL-specific marshaling (cost array translation, channel clamping). The entire system loads as standard NCCL plugins via environment variables, requiring zero NCCL modifications.

Verification and JIT layer. At load time, the BPF ELF is parsed and each program section is submitted to the static verifier. The verifier performs abstract interpretation over all possible execution paths, tracking register types, pointer provenance, and stack state at every instruction. It checks for: (1) memory safety—no null dereferences, no out-of-bounds accesses to the policy context or map values; (2) termination—no unbounded loops (all back-edges must have a proven bound); (3) stack safety—no writes beyond the 512-byte BPF stack; (4) helper authorization—only registered helper functions (`bpf_map_lookup_elem`, `bpf_map_update_elem`) can be called; and (5) context validity—reads and writes must be within the declared `policy_context` bounds. Programs that pass verification are JIT-compiled to native x86-64 code via LLVM. The JIT output executes at near-native speed, as our evaluation confirms (§5.1).

Map subsystem. eBPF maps serve dual roles: (1) intra-policy state (e.g., per-communicator channel counts, cumulative latency statistics) and (2) cross-plugin communication (profiler → tuner telemetry). Maps are typed: a hash map keyed by communicator ID stores per-communicator policy state; an array map stores global configuration. Map access from eBPF programs uses helper functions

(`bpf_map_lookup_elem`, `bpf_map_update_elem`) that are bounds-checked by the verifier. The verifier enforces that the return value of a lookup is checked for `NULL` before dereference—a property that prevents the most common class of crashes in native plugins (Scenario 1, §2.2).

Hot-reload engine. To update a policy, the operator provides a new BPF ELF file. The system verifies and JIT-compiles the new program, then atomically swaps the active function pointer using a compare-and-swap (CAS) operation. The old program remains valid until all in-flight calls complete; correctness of the full hot-reload path depends on: (1) the atomic CAS ensuring that the pointer swap is indivisible, (2) a quiescence protocol that retains the old program until in-flight calls drain (currently one epoch of calls), and (3) verification of the replacement program before the swap is attempted. If the new program fails verification, the swap is aborted and the old policy remains active—the system never enters an unverified state. This verify-then-swap design ensures that verification failure is always safe. The full reload cost (verification + JIT compilation + atomic swap) is approximately 11 ms; only the final swap step (0.309 µs) is on the latency-sensitive path.

3.4 Policy Programming Model

A NCCLPol policy is a BPF program that takes a *policy context* as input and writes its decisions into the context's output fields. The context structure is designed to expose NCCL's decision space while hiding internal implementation details:

Listing 1: Policy context structure exposing NCCL's collective decision space.

```
struct policy_context {
  /* inputs (read-only) */
  __u32 comm_id;    /* communicator ID */
  __u32 coll_type;  /* AllReduce, AllGather, ... */
  __u64 msg_size;   /* message size in bytes */
  __u32 n_ranks;    /* number of ranks */
  __u32 n_nodes;    /* number of nodes */
  /* outputs (write) */
  __u32 algorithm;  /* RING, TREE, COLLNET */
  __u32 protocol;   /* LL, LL128, SIMPLE */
  __u32 n_channels; /* parallelism degree */
};
```

The verifier ensures that policies only read from input fields and write to output fields. Any attempt to read beyond the context bounds or write to input fields is rejected at load time. This is enforced by treating the context pointer as a typed pointer with known offset bounds—the same mechanism used by the Linux kernel's BPF verifier for socket buffer access.

Example: Size-aware policy. The following policy selects algorithm and protocol based on message size, a common tuning heuristic derived from NCCL's own internal decision logic:

Listing 2: Size-aware tuning policy. The policy logic is 12 lines; the plugin boilerplate and runtime infrastructure are handled by the NCCLPol host plugin.

```
SEC("policy")
int size_aware(struct policy_context *ctx) {
  if (ctx->msg_size <= 4096) {
    ctx->algorithm = NCCL_ALGO_TREE;
    ctx->protocol = NCCL_PROTO_SIMPLE;
  } else if (ctx->msg_size <= 65536) {
    ctx->algorithm = NCCL_ALGO_TREE;
    ctx->protocol = NCCL_PROTO_LL;
  } else {
    ctx->algorithm = NCCL_ALGO_RING;
    ctx->protocol = NCCL_PROTO_LL;
  }
  return 0;
}
```

The policy logic itself is 12 lines. A native C++ plugin implementing the same decision logic requires approximately 200 lines, including `dlopen` boilerplate, manual error handling, and ABI glue code. The comparison is not entirely apples-to-apples: NCCLPol moves much of that complexity into the host plugin runtime (context marshaling, cost array translation, map management). The benefit is that *policy authors* need only express the decision logic, while the safety-critical infrastructure is shared and verified once.

Example: Closed-loop adaptation. A more sophisticated policy reads profiler-provided latency data from an eBPF map and adapts channel count dynamically:

Listing 3: Profiler-informed channel adaptation. The latency_map is written by the profiler plugin and read by the tuner policy.

```
1  SEC("policy")
2  int adaptive_channels(struct policy_context *ctx) {
3    __u32 key = ctx->comm_id;
4    struct latency_state *st =
5      bpf_map_lookup_elem(&latency_map, &key);
6    if (!st) { /* first call: initialize */
7      struct latency_state init = {.channels = 8};
8      bpf_map_update_elem(&latency_map, &key,
9                          &init, BPF_ANY);
10     ctx->n_channels = 8;
11     return 0;
12   }
13   /* adapt: high latency -> more channels */
14   if (st->avg_latency_ns > 1000000)
15     st->channels = min(st->channels + 1, 16);
16   else if (st->avg_latency_ns < 100000)
17     st->channels = max(st->channels - 1, 1);
18   ctx->n_channels = st->channels;
19   return 0;
20 }
```

The `latency_map` is a shared eBPF hash map. The profiler plugin writes updated latency measurements into this map on each profiler event callback; the tuner policy reads them on each `getCollInfo()` call. Note the mandatory null check on line 5: the verifier requires this check before dereferencing the map lookup result. This single verification rule eliminates the most common class of native plugin crashes.

This closed-loop design is difficult and error-prone with stock NCCL plugins because there is no built-in cross-plugin state-sharing mechanism. With native plugins, implementing this feedback loop would require: (1) a shared memory region manually allocated and

mapped by both plugins, (2) a custom locking protocol to ensure atomic access, (3) a type-unsafe cast from raw memory to a struct pointer, and (4) no verification that the profiler and tuner agree on the memory layout. While technically possible, this is exactly the kind of ad-hoc shared-state pattern that leads to subtle bugs. NCCLPol replaces all of this with a single typed map declaration.

3.5 Multi-Tenant Policy Differentiation

A single NCCLPol policy can differentiate behavior across communicators using the `comm_id` field in the policy context. This enables multi-tenant scenarios where different communicators (representing different tenants or job priorities) receive different tuning decisions from the same verified policy.

For example, an SLO-enforcing policy can assign aggressive parallelism (TREE, LL, 8 channels) to a latency-sensitive communicator (SLO $\leq$ 1 ms) while assigning conservative settings (RING, Simple, 1 channel) to a throughput-oriented communicator (SLO $\leq$ 10 ms). The per-communicator state is maintained in eBPF maps keyed by communicator ID, providing verified state separation between tenants' policy data. Note that this is *logical* state separation within a shared map, not process-level or hardware-enforced isolation; all policies still execute in the same address space.

This capability is significant because multi-tenant GPU clusters are becoming common, and different tenants' workloads have genuinely different communication characteristics. A dense transformer benefits from tree-based AllReduce with many channels; a mixture-of-experts model may prefer ring-based communication with fewer channels to reduce cross-node traffic. NCCLPol allows a single cluster-wide policy to express these per-tenant preferences safely.

3.6 Design Non-Goals

NCCLPol deliberately does *not* attempt to optimize collective algorithms themselves. Systems like MSCCL [2], TACCL [8], and MSCCL++ [7] generate efficient communication schedules for specific topologies; NCCLPol selects *among* NCCL's built-in algorithms at runtime. The two approaches are complementary: a custom MSCCL schedule can be activated by a NCCLPol policy that detects the appropriate conditions.

Similarly, NCCLPol does not replace NCCL's internal algorithm selection logic wholesale. NCCL's built-in tuning is the result of years of hardware-specific optimization. NCCLPol provides a *policy override* layer that operators use when they have domain-specific knowledge that NCCL's defaults do not capture (e.g., SLO constraints, multi-tenant priorities, workload-specific communication patterns).

4 IMPLEMENTATION

NCCLPol is implemented in approximately 2,500 lines of C/C++ (plugin host) and 500 lines of restricted C (policy programs). We describe key implementation decisions and the challenges encountered during real NCCL integration.

bpftime integration. We embed bpftime [11] as the userspace eBPF runtime. bpftime provides an LLVM-based JIT compiler, a PREVAIL-based [4] static verifier, and a map subsystem. We use bpftime's `bpftime_load_and_run` API to load BPF ELF objects and obtain JIT-compiled function pointers. The verifier runs in the same address space as the plugin, adding approximately 1–5 ms of one-time startup overhead (amortized over the lifetime of the training job). The LLVM JIT produces optimized x86-64 code that benefits from the same optimization passes as `clang -O2`, which is why JIT-compiled eBPF approaches native-code performance.

Plugin registration. NCCLPol registers itself as both a tuner and profiler plugin via NCCL's `ncclTunerPlugin_v3` and `ncclProfilerPlugin_v1` interfaces. The tuner plugin's `getCollInfo()` function constructs a `policy_context`, invokes the JIT-compiled eBPF function, and copies the output fields (algorithm, protocol, channels) into NCCL's result structure. The profiler plugin's event callbacks extract latency information and update the shared eBPF maps.

NCCL integration challenges. Real NCCL integration revealed several non-obvious requirements that are not documented in NCCL's plugin API specification:

Cost array semantics. NCCL's `getCollInfo()` does not accept a simple algorithm ID. Instead, it passes arrays of algorithm and protocol costs; the tuner must set costs to zero for the selected combination and to a large sentinel value (`1e9`) for all others. This indirect selection mechanism allows NCCL to fall back if the selected combination is not available for the current topology, but it means the policy's requested algorithm is a *preference*, not a mandate. Our integration correctly handles this by translating the eBPF policy's output fields into cost array entries.

Channel clamping. NCCL passes a `nChannels` parameter to `getCollInfo()` that represents the *maximum* number of channels the communicator can support. The tuner must respect this upper bound, not override it arbitrarily. Our native baseline layer clamps the policy's requested channel count to this limit.

Communicator identification. The communicator ID is not directly available in the tuner API as a simple integer. We derive it from the context pointer passed to the tuner, using a hash to produce a stable identifier across calls. This required careful handling to ensure consistency within a communicator's lifetime.

Hot-reload implementation. Policy hot-reload uses a `std::atomic<FuncPtr>` for the active policy function pointer. The reload path has three phases: (1) load and verify the new BPF program (~1–5 ms); (2) JIT-compile it to native code (~5–10 ms); (3) perform a `compare_exchange` on the atomic pointer (0.309 µs). The total reload cost observable by the operator is approximately 11 ms; only the final atomic swap is on the latency-sensitive path. The old function pointer is retained until a quiescence period (currently one epoch of calls) ensures no in-flight invocations reference it. In our stress test, we observe zero lost or corrupted calls across 400,000 concurrent invocations.

Native baseline for comparison. To provide a fair overhead comparison, we implemented a native C++ baseline that performs identical policy logic (size-based algorithm selection) without the eBPF layer. This baseline is compiled with the same optimization flags (`-O2`) and uses the same data structures. The overhead difference between the native baseline and eBPF policies directly measures the cost of the eBPF dispatch and JIT execution layers, isolated from any policy logic cost.

Table 1: CPU microbenchmark results (1M calls each). eBPF policies add 42–70 ns over native code. The overhead decomposes cleanly into dispatch, map lookup, and map update costs.

Policy	P50 (ns)	P99 (ns)	ΔP50 vs native
Native baseline	10	16	—
`noop`	51	61	+41
`size_aware_v2`	52	64	+42
`lookup_only`	63	74	+53
`lookup_update`	74	87	+64
`adaptive_channels`	75	88	+65
`slo_enforcer`	80	95	+70

Figure 2: Overhead Staircase *[placeholder]*

Stacked bar chart (6 bars, one per policy from Table 1), showing incremental overhead layers:

Native base (10 ns) | +41 ns eBPF dispatch | +11 ns map lookup | +11 ns map update

Y-axis: P50 latency (ns). Each bar annotated with P99 value.

Dashed line at 80 ns marks the most complex policy (slo_enforcer).

Data source: Table 1. Will be replaced with actual plot.

Figure 2: Overhead decomposition showing additive cost of each eBPF operation layer. The base dispatch cost (41 ns) dominates; each additional map operation adds a constant ∼11 ns. The clean staircase pattern indicates that overhead scales predictably with policy complexity.

5 EVALUATION

Our evaluation answers five research questions:

RQ1 What is the CPU overhead of eBPF policy execution relative to native code, and how does it decompose? (§5.1)
RQ2 Does the verifier correctly accept safe programs and reject all classes of unsafe programs? (§5.2)
RQ3 Can policies be hot-reloaded atomically without losing in-flight calls? (§5.3)
RQ4 Does NCCLPol work with real NCCL on real GPUs, and what is the end-to-end overhead? (§5.4)
RQ5 Can NCCLPol express non-trivial policy patterns—closed-loop adaptation, multi-tenant differentiation, contention response, and multi-collective coverage—within the eBPF programming model? (§5.5)

Testbed. CPU microbenchmarks run on an Intel platform with isolated cores to minimize scheduling noise. Each benchmark performs 1 million calls and reports P50 and P99 latencies. GPU experiments use a single NVIDIA RTX 5090 with CUDA 12.9 and NCCL 2.29.7. We emulate a 2-rank configuration on a single host using NCCL's `NCCL_HOSTID` mechanism over socket transport. While this configuration is modest in scale, it exercises the full plugin integration path: `getCollInfo()` is called with real topology information, real algorithm/protocol costs, and real channel constraints. The CPU-side policy overhead is independent of rank count.

5.1 RQ1: CPU Overhead

Table 1 reports CPU-side latency for 1 million calls to each policy. The results reveal a clean *overhead staircase* (Figure 2) that decomposes the total overhead into three additive components:

- **eBPF dispatch** (+41 ns): The fixed cost of entering the JIT-compiled eBPF function from the plugin's native code. This includes context structure setup (copying NCCL parameters into the `policy_context`), function pointer indirection, and callee-save register spill/restore. The `noop` policy—which returns immediately without accessing any context fields—isolates this cost precisely.
- **Map lookup** (+11 ns per operation): The `lookup_only` policy performs one `bpf_map_lookup_elem` call, adding 12 ns over `size_aware_v2` (which uses no maps). This cost includes hash computation on the key, hash table lookup, and pointer return.
- **Map update** (+11 ns per operation): The `lookup_update` policy adds one `bpf_map_update_elem` call, adding another 11 ns. This cost includes key hashing, bucket lookup, and value copy.

The clean staircase pattern means that overhead is *predictable*: a policy author can estimate the CPU cost of their policy by counting eBPF operations ($41 + 11n_{\text{lookup}} + 11n_{\text{update}}$ ns). This predictability is valuable for operators who need to reason about worst-case policy overhead.

Context: Why this overhead is negligible. The most complex policy (`slo_enforcer`) adds 70 ns over native code. For a 128 MB AllReduce that takes 175 µs on our GPU hardware, the 70 ns policy overhead is **0.04%** of collective latency. Even for the smallest collectives (1 KB messages, ∼5 µs on our hardware), the overhead is **under 1.5%**. This confirms our thesis from §2.3: NCCL's CPU-side decision path has a generous overhead budget that comfortably accommodates eBPF verification and execution.

Tail latency. P99 latencies track P50 closely (within 10–18 ns), indicating that eBPF JIT execution has minimal variance. This is expected: JIT-compiled code follows deterministic paths with no garbage collection, no dynamic allocation, and no system calls. The tight tail is important for training workloads where collective calls are serialized across ranks—a single high-latency policy decision delays all ranks.

Limitations. These microbenchmarks measure CPU-side overhead on isolated cores; production deployments with noisy neighbors, NUMA effects, and concurrent GPU DMA may exhibit higher variance. The overhead formula ($41 + 11n$ ns) is an empirical fit to our policies, not a guaranteed bound. Policies with larger map values or deeper nesting may deviate. We also do not measure the one-time JIT compilation cost (5–10 ms per policy) in this section; it is amortized over the job lifetime and reported in §5.3.

5.2 RQ2: Verifier Effectiveness

Table 2 shows the verifier's behavior on our test suite of 14 programs. The verifier uses PREVAIL-based [4] abstract interpretation to reason about all possible execution paths.

Table 2: Verifier results. All 7 safe policies are accepted; all 7 tested classes of unsafe programs are rejected with specific, actionable error messages. Each unsafe program targets a distinct bug class that would cause a crash or hang in a native plugin. These are author-constructed test cases, not drawn from a real bug corpus.

Program	Bug Class	Result
Safe policies (should accept)		
`noop`	—	ACCEPT
`size_aware`	—	ACCEPT
`size_aware_v2`	—	ACCEPT
`lookup_only`	—	ACCEPT
`lookup_update`	—	ACCEPT
`adaptive_channels`	—	ACCEPT
`slo_enforcer`	—	ACCEPT
Unsafe programs (should reject)		
`bad_lookup`	Null pointer deref	REJECT
`bad_oob_access`	OOB context read	REJECT
`bad_unregistered`	Illegal helper call	REJECT
`bad_stack_overflow`	Stack overflow	REJECT
`bad_infinite_loop`	Non-termination	REJECT
`bad_write_ctx`	Write to input field	REJECT
`bad_div_zero`	Division by zero	REJECT

Coverage of native-plugin failure modes. The seven unsafe programs are not arbitrary; each targets a specific bug class that we have observed (or that is commonly reported) in native NCCL plugins:

- `bad_lookup`: Uses a map lookup result without null checking. In a native plugin, this manifests as a `SIGSEGV` when the map entry does not exist (Scenario 1, §2.2).
- `bad_oob_access`: Reads beyond the declared context bounds. In a native plugin, this reads adjacent memory—potentially NCCL internal state—causing silent corruption (Scenario 2).
- `bad_infinite_loop`: Contains an unbounded loop. In a native plugin, this hangs the communicator indefinitely (Scenario 3).
- `bad_stack_overflow`: Allocates stack frames beyond the 512-byte BPF stack limit. In a native plugin, this corrupts the call stack.
- `bad_unregistered`: Calls a helper function not in the authorized set. This prevents policies from accessing arbitrary memory or invoking system calls.
- `bad_write_ctx`: Writes to a context field marked as read-only (an input field). This prevents policies from corrupting NCCL's internal state.
- `bad_div_zero`: Performs division by a variable that may be zero. The verifier tracks value ranges and rejects divisions where the divisor cannot be proven non-zero.

Safety comparison: crash vs. rejection. To validate that the verifier prevents real crashes, we implemented the equivalent of `bad_lookup` as a native C plugin—a deliberate null dereference in `getCollInfo()`. Loading this native plugin into NCCL produces an immediate `SIGSEGV`, crashing the process. The crash backtrace is:

```
Signal: SIGSEGV (address 0x0)
  in getCollInfo() at native_bad_plugin.so
```

Table 3: Hot-reload stress test. Atomic swap is sub-microsecond with zero lost or corrupted calls. Unsafe replacement is rejected and the old policy continues.

Metric	Value
Atomic swap window	0.309 µs
Active calls during test	400,000
Failed/lost calls	0
Corrupted results	0
Bad replacement rejected	Yes
Old policy preserved after rejection	Yes

The equivalent eBPF program is rejected at load time with:

```
VERIFIER REJECT: R0 is a pointer to
  map_value_or_null; must check != NULL
  before dereference at insn 7
```

This is the core safety proposition of NCCLPol: common bug classes that crash training jobs with native plugins are caught at load time with eBPF. The verifier error message is specific and actionable, pointing to the exact instruction and the required fix.

Limitations. Our verifier test suite consists of author-constructed programs designed to exercise specific bug classes. We do not claim coverage of all possible unsafe patterns; a real-world evaluation would require a corpus of actual plugin bugs, which to our knowledge does not exist for NCCL plugins. The verifier may have false negatives for patterns we did not test (e.g., complex aliased pointer arithmetic), and we did not systematically evaluate false positives—programs that are safe but rejected by the verifier's conservative analysis. The verifier itself (PREVAIL) and the JIT compiler (LLVM) are part of the trusted computing base; bugs in these components could undermine safety guarantees.

5.3 RQ3: Hot-Reload Integrity

Table 3 reports our hot-reload stress test. We continuously invoke `getCollInfo()` from multiple threads while concurrently swapping the active policy. The atomic swap completes in 0.309 µs, and we observe zero failed, lost, or corrupted calls across 400,000 invocations.

Full reload cost. The 0.309 µs figure is the atomic pointer-swap time alone. The full reload cost that an operator observes includes verification (~1–5 ms), JIT compilation (~5–10 ms), and the atomic swap—totaling approximately 11 ms. This cost is paid once per policy update and does not affect in-flight collective calls (verification and JIT run concurrently with normal policy execution; only the final swap briefly touches the hot path).

Why this matters. In production training jobs that run for days, operators frequently need to adjust tuning parameters—for example, to respond to changing network conditions, to accommodate a new tenant, or to fix a suboptimal configuration discovered during training. With native plugins, policy updates require restarting the NCCL communicator (and usually the entire training job), losing potentially hours of training progress. With NCCLPol, the operator

Table 4: End-to-end AllReduce latency on RTX 5090, 2 ranks over socket transport. Average overhead across all message sizes is <0.05 µs, within measurement noise.

Size	No Plugin (µs)	noop (µs)	size_aware (µs)	
128 MB	175.42	175.51	175.47	Avg. delta
				across all 10 message sizes: <0.05 µs

deploys a new BPF policy that is verified, JIT-compiled, and atomically swapped with ∼11 ms of total reload time. The training job continues uninterrupted.

Safety under operator error. When we attempt to hot-reload an *unsafe* policy (one that fails verification), the system rejects the replacement and preserves the currently-active safe policy. The operator receives a verification error message; the training job is unaffected. This fail-safe property is important because it means that even operator mistakes during policy updates cannot crash or degrade a running job.

Limitations. Our stress test uses a single process with multiple threads; we do not test hot-reload across multiple NCCL ranks or under real GPU collective load. The quiescence protocol (retaining the old function pointer for one epoch) is a simple heuristic; a more rigorous approach would use epoch-based reclamation or hazard pointers. We also do not test repeated rapid reloads (e.g., hundreds of swaps per second), which could accumulate deferred function pointers.

5.4 RQ4: End-to-End GPU Overhead

We integrate NCCLPol with NCCL 2.29.7 running a 2-rank AllReduce benchmark on a single RTX 5090 with socket transport. Each rank calls `getCollInfo()` 495 times across 10 message sizes (1 KB to 128 MB).

Latency overhead. Table 4 shows end-to-end GPU-measured collective latency. At 128 MB, the `noop` policy adds 0.09 µs and `size_aware_v2` adds 0.05 µs—both within measurement noise. The average overhead across all message sizes is below 0.05 µs. This confirms our thesis that NCCL's CPU-side decision path is sufficiently latency-tolerant to absorb eBPF overhead without measurable impact on GPU communication performance.

Why the overhead vanishes at end-to-end. The 42–70 ns measured in CPU microbenchmarks is invisible at the GPU end-to-end level because (1) the CPU policy decision occurs before the GPU collective launch, and the two are not on the critical path simultaneously; (2) NCCL's internal processing between `getCollInfo()` return and actual GPU kernel launch adds microseconds of overhead that dwarfs the eBPF cost; and (3) GPU collective execution itself dominates total latency.

Limitations. Our end-to-end experiment uses a 2-rank, single-node configuration over socket transport. While this validates the full plugin integration path and confirms that CPU-side overhead is invisible at the GPU level, it does not exercise multi-node topologies, InfiniBand/RoCE transports, or large rank counts where NCCL's internal behavior differs significantly. The CPU-side policy overhead is rank-count-independent (each rank calls `getCollInfo()` locally), but we cannot rule out interactions at scale (e.g., with NCCL's internal thread synchronization or transport setup).

Figure 3: Policy Adoption by Message Size *[placeholder]*

Heatmap: rows = message sizes, columns = algorithm×protocol. Cell shading indicates policy request vs. NCCL's actual selection:

1–2 KB:	TREE + SIMPLE
4–32 KB:	TREE + LL
64–512 KB:	RING + LL
1 MB:	RING + SIMPLE

10/10 sizes match on our test topology. Will be replaced with actual plot.

Figure 3: Algorithm and protocol adoption by message size. NCCL accepts the policy's recommendation for all 10 message sizes tested, confirming that NCCLPol policies have effective influence over NCCL's algorithm selection via the cost-array mechanism on our test topology.

Policy adoption verification. Our `size_aware_v2` policy requests specific algorithm/protocol combinations based on message size. As shown in Figure 3, NCCL adopted the policy's recommendation for all 10 message sizes on our test topology:

- 1–2 KB: TREE + SIMPLE (tree reduces latency for small messages; Simple avoids LL protocol overhead when messages fit in a single packet)
- 4–32 KB: TREE + LL (LL protocol reduces latency for medium messages where its inline-data optimization is effective)
- 64–512 KB: RING + LL (ring algorithm achieves better bandwidth utilization for larger messages)
- 1 MB: RING + SIMPLE (largest messages benefit from Simple protocol's higher bandwidth)

This 10/10 adoption rate demonstrates that NCCLPol policies have effective influence over NCCL's algorithm selection via the cost-array mechanism. We note that the cost-array interface expresses a *preference*, not a mandate: NCCL may override the policy if the requested algorithm/protocol combination is unavailable for the current topology. On our single-node test topology, all requested combinations were available, so we observed 100% adoption. Different topologies or transport configurations may yield lower adoption rates, which is by design—the cost-array fallback mechanism ensures NCCL never uses an unsupported combination.

5.5 RQ5: Policy Expressiveness

We evaluate four policy capabilities that go beyond simple algorithm selection, demonstrating that NCCLPol's eBPF-based approach enables policy patterns that are difficult and error-prone with native plugins. We frame this section as a *policy expressiveness demonstration*: we show that the eBPF programming model can express these patterns, not that the resulting parameter changes improve end-to-end training outcomes. Evaluating outcome improvements (latency, throughput, job completion time) on production workloads is important future work.

Figure 4: Multi-Tenant Differentiation *[placeholder]*

Side-by-side comparison of two communicators under the same policy binary, differentiated by comm_id:

	Comm 1 (SLO 1ms)	**Comm 2 (SLO 10ms)**
Algorithm	TREE	RING
Protocol	LL	SIMPLE
Channels	8	1
Strategy	Aggressive	Conservative

Shows parameter differentiation, not outcome improvement. Will be replaced with actual plot.

Figure 4: Multi-tenant differentiation. The same `slo_enforcer` policy produces different algorithm/protocol/channel selections for two communicators with different SLO requirements, using per-communicator state in eBPF maps.

Figure 5: Contention Adaptation Time Series *[placeholder]*

Line plot: X-axis = collective call index, Y-axis = channel count (1–16).

Phase 1 – Baseline (low latency): channels rise to 12
Phase 2 – Contention injected (via map write): channels fall to 2
Phase 3 – Recovery (low latency): channels recover to 12

Shaded region indicates synthetic contention injection period (manual map write, not real interconnect contention). Will be replaced with actual plot.

Figure 5: Dynamic channel adaptation under changing contention. The policy increases channels when latency is low (more parallelism) and decreases channels when contention is detected (reduce interference). Recovery is symmetric with ramp-up, demonstrating stable feedback control through eBPF map state.

5a: Closed-loop profiler-to-tuner adaptation. We deploy the `adaptive_channels` policy alongside the profiler plugin. The profiler measures real collective latency (by observing NCCL's profiler event timestamps) and writes it to a shared eBPF map. The tuner reads this map and adjusts channel count:

- **With profiler feedback**: channels adapt from 8 → 9 → 10 over successive collectives, based on observed latency (first measured sample: 9,264,480 ns at rank 0).
- **Without profiler feedback**: channels remain static at 8 → 8 → 8 across all collectives.

The difference confirms that the profiler data is flowing through the eBPF map to the tuner and actually affecting decisions. This closed-loop control has no built-in support in NCCL's native plugin architecture, where the tuner and profiler plugins are loaded independently with no shared state mechanism. An operator using native plugins who wanted this behavior would need to implement ad-hoc shared memory between two separate `.so` files loaded by the same process—a fragile and error-prone undertaking that NCCLPol's typed map abstraction avoids entirely.

5b: Multi-tenant differentiation. We deploy the `slo_enforcer` policy with two communicators in the same process, simulating a multi-tenant scenario where each communicator has a different SLO (Figure 4):

- Communicator 1 (SLO ≤ 1 ms): receives TREE / LL / 8 channels (aggressive, latency-optimized). The policy selects tree topology and low-latency protocol to minimize per-collective time.
- Communicator 2 (SLO ≤ 10 ms): receives RING / Simple / 1 channel (conservative, throughput-optimized). The policy selects ring topology and high-bandwidth protocol to maximize throughput within the relaxed SLO.

The same verified policy binary produces differentiated behavior based on per-communicator state stored in eBPF maps keyed by `comm_id`. This demonstrates a multi-tenant QoS capability that would require complex, error-prone shared state management in a native plugin. We note that this experiment shows parameter differentiation, not that the differentiated parameters improve end-to-end QoS for the respective tenants—outcome evaluation on multi-tenant workloads is future work.

5c: Contention-aware adaptation. Figure 5 shows the `adaptive_channels` policy's response to simulated contention phases. During the baseline phase (low observed latency), the policy ramps channel count up to 12 to maximize parallelism. When contention is injected (simulated via elevated latency values in the profiler map), the policy reduces channels to 2 to minimize interference with other workloads sharing the interconnect. When contention subsides, channels recover to 12.

This reactive behavior emerges naturally from the map-based feedback loop—no special contention-detection mechanism is needed. The policy simply responds to latency observations in the shared map. The symmetric ramp-up/ramp-down behavior suggests that the feedback loop is stable under this synthetic contention pattern (no oscillation observed). However, we caution that contention was injected by manually writing elevated latency values into the profiler map, not by generating actual interconnect contention. Stability under real contention patterns—which may be bursty, asymmetric, or correlated across communicators—remains to be validated. Per-communicator state in eBPF maps ensures that each communicator's adaptation is logically independent.

5d: Multi-collective coverage. We verify that NCCLPol policies work across all major collective types supported by NCCL: AllReduce, AllGather, ReduceScatter, and Broadcast. The policy context's `coll_type` field enables collective-specific tuning strategies. In our tests:

- **AllReduce** shows distinct TREE algorithm selection for small messages (where tree topology's logarithmic latency scaling benefits small collectives), transitioning to RING for larger messages.
- **AllGather, ReduceScatter, Broadcast** default to RING algorithm across all message sizes, as these collectives have different communication patterns where ring topology is generally preferred.

This demonstrates that a single NCCLPol policy can express collective-type-aware tuning strategies, covering the full range of NCCL's collective operations.

6 DISCUSSION

Current limitations. NCCLPol currently targets NCCL's tuner and profiler plugin interfaces. The net plugin interface, which controls network device selection and data transfer, involves more complex state (connection handles, memory registrations, RDMA queue pairs) that does not map cleanly to eBPF's restricted programming model. Extending NCCLPol to cover net plugin decisions is future work that may require helper function extensions to the eBPF runtime for network resource management.

Our evaluation uses a 2-rank configuration on a single node. While this is sufficient to validate the plugin integration and measure overhead (the CPU-side policy path is rank-count-independent), evaluating multi-node scenarios with InfiniBand transport and hundreds of ranks remains important future work. We expect the CPU overhead results to hold because `getCollInfo()` is called locally at each rank, but the policy effectiveness of adaptive channel selection may differ at scale.

The current policy programming model requires authors to write in restricted C and compile to BPF bytecode. While this is familiar to systems programmers experienced with Linux eBPF, it presents a barrier for ML engineers who want to express simple tuning preferences. A higher-level policy specification language (e.g., declarative SLO constraints compiled to BPF) would improve usability.

eBPF expressiveness boundaries. Not all conceivable NCCL policies can be expressed in eBPF. Policies that require floating-point arithmetic (e.g., gradient-aware bandwidth allocation), dynamic memory allocation (e.g., building communication graphs), or access to GPU memory (e.g., inspecting tensor contents) exceed eBPF's execution model. We argue that these limitations are acceptable for the policy decision layer: the question "which algorithm and protocol should this collective use?" is inherently a function of metadata (message size, rank count, topology, historical latency), not tensor content. Policies that need GPU-side information can receive it through profiler-populated maps, maintaining the verified execution boundary.

Broader applicability. The eBPF-for-extensions pattern demonstrated by NCCLPol may be applicable beyond NCCL. Other libraries with native-code plugin interfaces—MPI implementations (OpenMPI, MPICH), distributed training frameworks (PyTorch FSDP, DeepSpeed), RDMA middleware—face similar safety gaps. The observation that GPU-adjacent CPU-side decision paths have generous latency budgets for verified execution may generalize to other parts of the accelerator software stack, though each new domain would require its own evaluation of overhead tolerance and action-space expressiveness.

EIM integration. The Extension Interface Model (EIM) [1] provides a formal capability specification for eBPF extensions, defining what maps, helpers, and context fields each extension point can access. Integrating EIM with NCCLPol would enable fine-grained access control—e.g., a policy could be granted read access to latency maps but denied access to network device maps—further strengthening the safety guarantees beyond what the verifier alone provides. EIM would also enable formal reasoning about policy composition: can two independently-verified policies be safely composed when sharing maps?

7 RELATED WORK

We position NCCLPol against three bodies of work: collective communication optimization, eBPF-based system extensibility, and GPU runtime policy.

Collective communication optimization. MSCCL [2] introduced programmable collective communication schedules synthesized from topology-aware algorithms. TACCL [8] extended this approach with communication sketches that guide synthesis toward efficient schedules. Both systems produce *static, offline* plans: they optimize the communication schedule itself but do not address *runtime policy selection*—the question of which schedule to use for a given collective call in a given context. NCCLPol operates at this runtime policy layer, and can serve as the mechanism that activates MSCCL/TACCL schedules based on dynamic conditions.

AutoCCL [3] automates collective tuning through search-based optimization, testing different algorithm/protocol combinations and selecting the best. AutoCCL's tuning decisions are implemented as native code with no safety guarantees. NCCLPol provides the same decision capability (selecting algorithms and protocols per collective) but with verification, state separation, and hot-reload—properties that AutoCCL's native approach does not address.

MSCCL++ [7] provides a GPU-driven communication stack with fine-grained primitives (put/get, atomic operations). Its contribution is at the *mechanism* layer—how data is moved between GPUs—while NCCLPol operates at the *policy* layer—deciding which mechanism to use. The two are complementary: a NCCLPol policy could select between MSCCL++ channels based on message size or topology.

eBPF for systems extensibility. XRP [12] embeds eBPF programs in the kernel's NVMe completion path, allowing applications to push storage resubmission logic close to the device. Like NCCLPol, XRP exploits the insight that certain system paths (storage completion, collective decision) are latency-tolerant enough to accommodate eBPF overhead. However, XRP operates in kernel space on a storage I/O path, while NCCLPol operates in userspace on a GPU communication path. The domains are different, but the architectural pattern—identify a latency-tolerant extension point, embed eBPF for safety—is the same.

Electrode [13] uses kernel eBPF to accelerate distributed protocols (Paxos, sequencer) by executing protocol logic in the network stack. eTran [5] applies eBPF to transparent network function composition for RDMA transports. PageFlex [10] extends eBPF to OS page management, allowing applications to customize page allocation and migration policies. These systems collectively demonstrate that eBPF's verification model enables safe extensibility in previously-closed subsystems. NCCLPol extends this line of work to a new domain—GPU collective communication—that has not been addressed by prior eBPF systems.

bpftime [11] provides the userspace eBPF runtime substrate that NCCLPol builds upon. bpftime's key contribution is enabling eBPF execution outside the kernel, using LLVM JIT and the PREVAIL verifier [4]. NCCLPol validates bpftime's design in a challenging new application domain: inside a performance-critical GPU communication library where sub-microsecond overhead is essential.

GPU runtime policy and scheduling. Prior work on GPU scheduling and resource management has explored policies for GPU kernel

scheduling, memory management, and multi-tenancy. However, these systems operate at the GPU device level (scheduling compute kernels on SMs) or at the framework level (scheduling training operators). NCCLPol operates at the *communication runtime* level—between the framework and the network—a layer that has received little attention from the policy and extensibility perspective.

Alternative safe-extension mechanisms. Several alternative approaches could provide safe extensibility for NCCL plugins, each with different tradeoffs:

WebAssembly (Wasm) provides a general-purpose sandboxed execution environment with linear memory isolation. Wasm is more expressive than eBPF (supporting floating-point, dynamic memory, complex control flow) but lacks eBPF's static verification of termination and memory safety before execution. Wasm's runtime overhead is typically higher than JIT-compiled eBPF due to bounds checks on memory accesses that eBPF's verifier eliminates statically. For NCCL policy decisions, which are inherently simple (selecting from small enumerations), Wasm's extra expressiveness is unnecessary, while eBPF's verification guarantees are directly valuable.

Out-of-process tuning services eliminate the in-process isolation concern entirely by running policy logic in a separate process and communicating decisions via IPC (pipes, sockets, shared memory). This provides strong isolation but adds microseconds of IPC latency per policy decision—acceptable for some use cases but an order of magnitude more than eBPF's 42–70 ns in-process overhead. Out-of-process approaches also complicate state sharing (e.g., profiler-to-tuner feedback requires cross-process communication).

Restricted declarative DSLs (e.g., a YAML-based policy specification compiled to lookup tables) are safe by construction but limited in expressiveness: they cannot express stateful policies, feedback loops, or conditional logic beyond simple thresholds. eBPF occupies a middle ground—more expressive than declarative DSLs (loops, maps, conditionals) while still verifiable.

Hardware-assisted isolation (Intel MPK, ARM MTE) can enforce memory isolation between extensions at near-zero overhead. However, MPK provides isolation without verification—a policy can still crash within its own domain or enter infinite loops. eBPF's verifier catches these bugs at load time. MPK and eBPF verification are complementary; future work could combine them for defense in depth.

We chose eBPF because it provides both verification *and* efficient execution in a single model, the BPF toolchain (clang, verifier, JIT) is mature and widely deployed, and the map abstraction directly addresses NCCL's cross-plugin state-sharing gap.

8 CONCLUSION

We presented NCCLPol, a system that transforms NCCL's unsafe native plugin extension points into verified, composable policy hooks by embedding a userspace eBPF runtime inside the plugin interface. Our key insight is that NCCL's CPU-side collective decision path is sufficiently latency-tolerant to absorb eBPF's verification and execution overhead, making the safety–performance tradeoff favorable. NCCLPol adds 42–70 ns of CPU overhead per collective decision (0.04% of collective latency), rejects common classes of unsafe programs at load time, enables atomic policy hot-reload (~11 ms total, 0.309 µs on the hot path) without call loss, and supports closed-loop profiler-to-tuner adaptation through typed eBPF maps. The system requires zero modifications to NCCL, operating entirely within its stock plugin ABI.

Beyond NCCL, the architectural pattern demonstrated by NCCLPol—embedding verified eBPF execution in latency-tolerant CPU-side decision paths—may be applicable to other parts of the accelerator software stack where native-code extensions pose similar safety risks.

REFERENCES

[1] bpftime contributors. 2025. EIM: Extension Interface Model Specification. https://eunomia.dev/bpftime/EIM/spec/.

[2] Zhihao Cai et al. 2023. MSCCL: Programmable Communication Schedules for GPU Clusters. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*

[3] Zhihao Cai et al. 2024. AutoCCL: Automated Collective Communication Tuning for GPU Clusters. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC).*

[4] Elazar Gershuni, Nadav Amit, et al. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI).*

[5] Bing Liu et al. 2024. eTran: Efficient and Extensible Transparent Network Functions. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*

[6] NVIDIA. 2024. NCCL: NVIDIA Collective Communications Library. https://github.com/NVIDIA/nccl.

[7] Mehrdad Salehi et al. 2025. MSCCL++: A GPU-driven Communication Stack for Scalable AI Training. In *Proceedings of the 20th European Conference on Computer Systems (EuroSys).*

[8] Aashaka Shah et al. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*

[9] Nelson Billing Vieira et al. 2020. Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications. *Comput. Surveys.*

[10] Xueyuan Wang et al. 2025. PageFlex: eBPF-Based Programmable Page Management. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS).*

[11] Tong Yu, Yusheng Zheng, et al. 2025. bpftime: Userspace eBPF Runtime for Uprobe, Syscall and Kernel-Function Tracing. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*

[12] Yuhong Zhong, Haoyu Zhu, et al. 2022. XRP: In-Kernel Storage Functions with eBPF. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*

[13] Yu Zhou et al. 2023. Electrode: Accelerating Distributed Protocols with eBPF. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*